## Лабораторная работа №1

# Спецификация и архитектурное проектирование многоагентной системы

Курс «Теория многоагентных систем» · 12 ноября 2026 года

**Выполнил:** Мозговой Кирилл Сергеевич
**Группа:** Б24-517

### Тема и цель

Распределение заказов между автономными курьерами. Цель работы — сформулировать задачу, определить агентов, их знания, состояния и полномочия, спроектировать взаимодействие и обосновать архитектуру, пригодную для исследований в ЛР2–ЛР4.

### 1. Прикладная задача

Служба доставки получает заказы с координатами получения и доставки, массой и желательным сроком выполнения. Курьеры различаются местоположением, скоростью и грузоподъёмностью. Система должна назначать выполнимые заказы, отслеживать исполнение и сохранять журнал обмена сообщениями.

Курьер представлен агентом, поскольку располагает локальным состоянием и собственным правилом принятия задания. Назначение диспетчера является предложением: занятый курьер или курьер с недостаточной грузоподъёмностью отклоняет его. Это позволяет явно отделить координацию от исполнения.

### Постановка и критерии

Вход: множество заказов O и множество курьеров C. Заказ o = (id, r, p, d, w, D): идентификатор, время поступления, точка получения, точка доставки, масса и дедлайн. Профиль курьера c = (id, $x_0$, K, v): начальная позиция, грузоподъёмность и скорость.

Выход: назначение исполнителей, состояния заказов, времена доставки и трасса сообщений. Обязательные ограничения: один активный заказ на курьера; масса заказа не больше грузоподъёмности; один исполнитель на заказ; доставка только после получения. Опоздание допускается и учитывается отдельно.

### Допущения модели

Город — целочисленная сетка; расстояние — манхэттенское. Время дискретное, погрузка и выгрузка мгновенные. Курьеры движутся сначала по оси x, затем по y. Частичное обслуживание и передача груза между курьерами не предусмотрены. Все агенты работают в одном процессе через надёжную FIFO-шину; задержки и потери связи в ЛР1 отсутствуют.

# 2. Формальное описание агентов

Система задаётся как M = (A, O, S, Act, Msg, T), где A = {source, dispatcher, courier_1, courier_2, courier_3}; S — совокупность локальных состояний; Act — действия агентов; Msg — типизированные сообщения; T — дискретный шаг моделирования.

### Источник заказов — source

**Цель:** передать предусмотренные сценарием заявки в момент r. **Знания:** список заказов и времена их появления. **Состояние:** текущий момент симуляции; перечень задан статически. **Действие:** отправить REQUEST диспетчеру. **Полномочия:** создавать заказы, но не выбирать исполнителя и не менять их статус. **Средства:** сценарий и исходящий канал source → dispatcher.

### Диспетчер — dispatcher

**Цель:** назначать ожидающие заказы доступным исполнителям, уменьшая оценку времени маршрута. **Знания:** реестр заказов, статические профили курьеров, последние STATUS, ожидаемые ответы, подтверждённые назначения и времена доставки. **Состояние:** waiting / offered / assigned / in_transit / delivered для каждого заказа. **Действия:** выбрать кандидата, зарезервировать его, отправить ASSIGN, обработать ответ и обновить реестр.

**Полномочия:** предлагать заказы и вести общий реестр. Диспетчер не меняет напрямую позицию, груз или состояние курьера. **Средства:** очередь заказов, функция оценки маршрута и каналы dispatcher ↔ courier_i. Сведения об исполнении поступают только из сообщений.

### Курьеры — courier_i

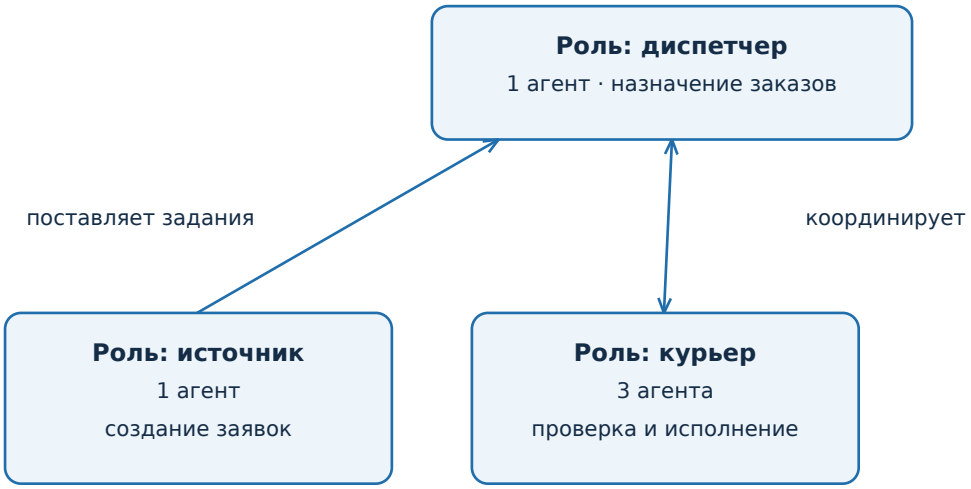
**Локальная цель:** выполнить принятое задание без превышения собственной грузоподъёмности и без двух одновременных заказов. В ЛР1 агенты кооперативны, денежная полезность не вводится. **Знания:** текущая позиция, скорость, грузоподъёмность, принятый заказ и множество ранее принятых идентификаторов. Курьер не знает очереди остальных агентов.

**Состояния:** idle, to_pickup, to_destination. **Действия:** проверить предложение, принять или отклонить его, переместиться, получить и доставить заказ, сообщить состояние. **Полномочия:** управлять только собственным исполнением. **Средства:** локальный маршрут и канал courier_i ↔ dispatcher.

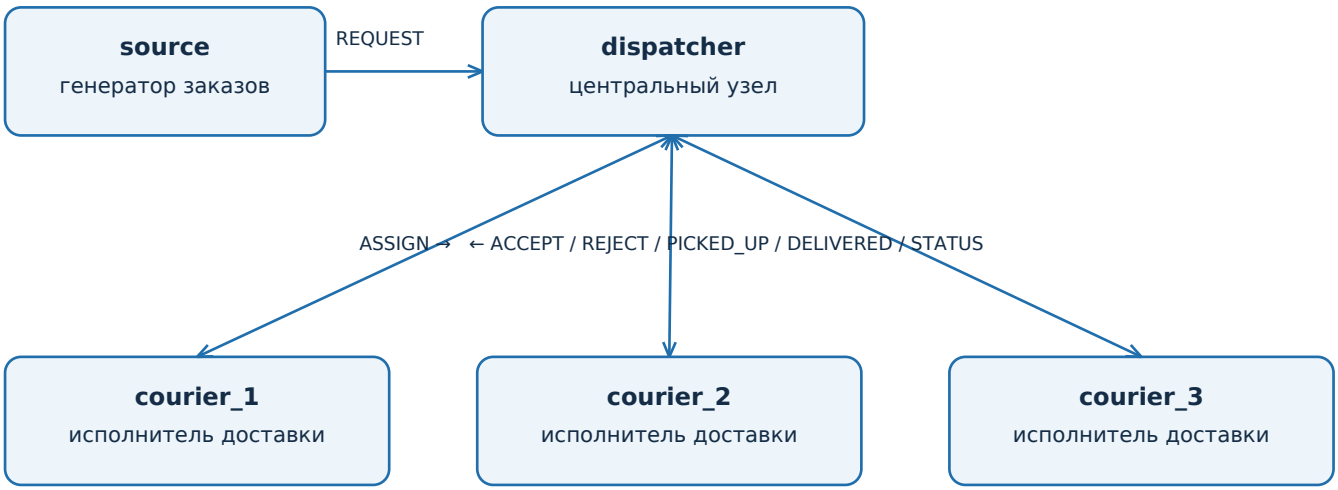
| Агент | Стартовая позиция | Грузоподъёмность | Скорость |
|---|---|---|---|
| courier_1 | (0, 0) | 3 ед. массы | 1 клетка/такт |
| courier_2 | (8, 0) | 6 ед. массы | 1 клетка/такт |
| courier_3 | (4, 8) | 4 ед. массы | 2 клетки/такт |

# 3. Архитектура и коммуникации

## Диаграмма ролей




## Граф коммуникаций




## Обоснование выбора

Выбрана иерархическая MAC с централизованной координацией и автономными исполнителями. Для небольшой службы это даёт простой реестр назначений и позволяет исключить конкурирующие назначения резервированием. Различие скоростей и грузоподъёмностей учитывается явно; решения исполнителей остаются локальными.

Цена решения — единая точка отказа и концентрация сообщений у диспетчера. Координация пока не распределённая; отказ центрального узла, устаревшие сведения и недостоверные ответы требуют отдельных механизмов. Эти ограничения задают предмет следующих работ.

Шина Bus — инфраструктура передачи, не отдельный агент: у неё нет цели выбора исполнителя. Классы агентов обмениваются сообщениями и не вызывают методы изменения состояния друг друга. Планировщик симуляции только упорядочивает обработку и движение.

# 4. Протокол и правила взаимодействия

| Сообщение | Направление | Содержание и эффект |
|---|---|---|
| REQUEST | source → dispatcher | Все поля заказа; новый id регистрируется как waiting. |
| ASSIGN | dispatcher → courier | Все поля заказа; диспетчер ставит offered и резервирует кандидата. |
| ACCEPT | courier → dispatcher | id заказа; при совпадении ожидаемого отправителя фиксируются assigned и исполнитель. |
| REJECT | courier → dispatcher | id, позиция, idle; снимается ожидание ответа, заказ возвращается в waiting. |
| PICKED_UP | courier → dispatcher | id; переход assigned → in_transit от подтверждённого исполнителя. |
| DELIVERED | courier → dispatcher | id; переход in_transit → delivered и фиксация времени. |
| STATUS | courier → dispatcher | Позиция и idle после такта активного курьера; обновление знаний диспетчера. |

### Выбор исполнителя

Заказы рассматриваются по (время поступления, id). Кандидаты — свободные курьеры с $K \geq w$. Для каждого вычисляется оценка $\tau = (|x - p|_1 + |p - d|_1) / v$. Выбирается минимум $\tau$, при равенстве — лексикографически меньший id. Это локальная эвристика без доказанной глобальной оптимальности; дедлайн в приоритезацию не входит.

### Переходы и защита ограничений

Заказ: waiting → offered → assigned → in_transit → delivered; при отказе offered → waiting. Курьер: idle → to_pickup → to_destination → idle. Принять можно только новый id при idle и $w \leq K$. Резервирование кандидата до отправки ASSIGN не позволяет предложить ему второй заказ в том же цикле. Повтор ASSIGN для уже принятого id не запускает исполнение повторно.

Ответ ACCEPT/REJECT обрабатывается только от ожидаемого кандидата. PICKED_UP и DELIVERED учитываются от закреплённого исполнителя в допустимом состоянии. Это проверки логической согласованности, а не аутентификация: криптографической проверки отправителя нет.

При отсутствии кандидата заказ остаётся waiting. В надёжной синхронной модели ответ поступает в том же цикле обработки шины; таймаут назначения не реализован. В моделях с потерями сообщений необходимо добавить таймаут, повтор и идентификатор попытки.

# 5. Прототип и проверка архитектуры

## Реализация

Прототип написан на Python 3.10+ со стандартной библиотекой. В src/simulation.py явно реализованы Source, Dispatcher, Courier и Bus. Шаг: поступление заявок → обработка шины → назначение → обработка ответов → движение → обработка событий. Доставка фиксируется в конце такта; даже нулевой маршрут завершается не раньше конца такта назначения.

В одном такте курьер проходит не больше v клеток, включая оба участка маршрута; оставшийся после получения бюджет движения может использоваться для доставки. Заказ, завершённый в конце такта, освобождает курьера для назначения в следующем такте.

## Сценарии и фактические результаты

Проверены три детерминированных сценария при политике nearest и пределе 120 тактов. Normal: 6 заказов с поступлением в такты 0–4. Overload: 18 заказов в такт 0 с дедлайном 10. Capacity: normal плюс заказ HEAVY массой 9 при максимальной грузоподъёмности 6. Полные входные данные и трассы вынесены в приложения.

| Сценарий | Доставлено | Такты | Поздно | Ср. время | Сообщ. | Путь |
|---|---|---|---|---|---|---|
| Normal | 6/6 | 12 | 0 | 5.667 | 60 | 38 |
| Overload | 18/18 | 90 | 16 | 37.889 | 281 | 235 |
| Capacity | 6/7 | 120 | 0 | 5.667 | 61 | 38 |

Среднее время = среднее (время доставки − время поступления) только по доставленным заказам. «Поздно» — число доставок строго после дедлайна. «Сообщ.» включает STATUS; «Путь» — сумма пройденных клеток. «Такты» — длительность запуска, а не средняя задержка.

## Интерпретация

В обычном режиме все 6 заказов доставлены за 12 тактов без опозданий. При перегрузке все 18 доставлены, но 16 опоздали: отсутствие учёта дедлайнов и ограниченная производительность существенны. В сценарии capacity шесть обычных заказов завершены, HEAVY остаётся waiting до предела. Система сохраняет ограничение по массе, но не умеет эскалировать невыполнимую заявку.

## Проверки корректности

Пять тестов unittest завершились успешно. Проверяются завершение выполнимых сценариев для двух политик; единственность принятия и ограничение массы; изоляция невыполнимого заказа; отказ занятого/перегруженного агента и повтор ASSIGN; будущая заявка с нулевым маршрутом; детерминизм и причинный порядок REQUEST → ASSIGN → ACCEPT → PICKED_UP → DELIVERED. Это функциональные проверки прототипа, не исследование надёжности ЛР4.

# 6. Развитие системы и выводы

## Пригодность для последующих лабораторных

| Работа | Планируемое исследование |
|---|---|
| ЛР2. Распределение | Сравнить nearest и round_robin на одинаковых входах, затем при необходимости реализовать контрактную сеть. Менять интенсивность поступления и состав курьеров; измерять задержку, опоздания, сообщения и путь. Две первые политики уже переключаются в прототипе; сравнительное исследование в ЛР1 не проводилось. |
| ЛР3. Конфликты | Ввести ограниченное число мест погрузки в одном депо. Курьеры конкурируют за интервалы; сравнить обслуживание без переговоров и с запросом временного слота, приоритетом по дедлайну и ростом приоритета ожидания. Измерять задержку и справедливость доступа. Пока не реализовано. |
| ЛР4. Надёжность | Моделировать отказ курьера, потерю ответа, устаревший STATUS и неверное сообщение о завершении. Добавить таймауты, идентификаторы попыток, контроль переходов и переназначение; сравнить трассы и долю завершённых заказов. Пока не реализовано. |

## Ограничения результата

Рассматривается логическая модель, а не промышленная служба доставки: нет дорог с препятствиями, реального времени, сетевых процессов, аутентификации и оплаты. Автономия курьеров ограничена принятием и исполнением заказа; переговоры об условиях не реализованы. Реестр seen растёт без очистки, что приемлемо для небольших сценариев, но требует политики хранения при длительной работе.

## Вывод

Сформулирована прикладная задача и спроектирована МАС из пяти агентов трёх ролей. Определены знания, цели, состояния, действия, полномочия и каналы; построены диаграмма ролей и граф коммуникаций. Центральная координация обеспечивает обозримый протокол назначения, а локальные проверки курьеров сохраняют ограничения исполнения. Работоспособность архитектуры подтверждена прототипом и воспроизводимыми запусками. Структура допускает расширение для ЛР2–ЛР4.

## Источник требований и приложения

Источник: «Рекомендации по выполнению лабораторных работ по курсу “Теория многоагентных систем”», файл «Лабораторные-МАС.pdf», с. 1–2. Требования к ЛР1 — с. 2; общие требования к реализации и отчёту — с. 1.

Исходный код и демонстрационные материалы размещены в репозитории GitHub:
https://github.com/KirillMozgovoy/mas/tree/main/lab1
Состав: src/simulation.py, tests/test_simulation.py, demo/index.html и demo/*.json. Инструкции запуска — README.md. В настоящий отчёт исходный код и демонстрационные трассы не включены.